

CHORD

Engineering Specification & Architectural Manual

CONFIDENTIAL | INTERNAL ENGINEERING USE ONLY

Generated: 2026-02-21

1. Architectural Blueprint

Chapter 1: Architectural Blueprint

System Topology

Upon examining the provided data, it is evident that the system is comprised of three primary components: `Tracker.java`, `TrackerIntf.java`, and `ChordIntf.java`. These components form the foundation of the system's architecture, and their relationships will dictate the overall structure and behavior of the application.

Core Orchestration Pattern

Analysis of the system's components reveals a Service-Oriented Architecture (SOA) pattern, with a focus on interface-driven design. The presence of `TrackerIntf.java` and `ChordIntf.java` indicates that these interfaces serve as contracts for the `Tracker.java` component, defining the expected behavior and interactions. This pattern enables loose coupling, allowing for greater flexibility and maintainability in the system.

Component Analysis

1. **Tracker.java**: This component serves as the primary entry point for the system. Its downstream impact is significant, as it will likely interact with the `TrackerIntf.java` and `ChordIntf.java` interfaces to perform its functions. The absence of dependencies and symbols suggests that this component is designed to operate independently, with minimal external influences.
2. **TrackerIntf.java**: As an interface, this component defines the contract for the `Tracker.java` component. Its presence ensures that the `Tracker.java` component adheres to a specific set of behaviors and interactions, promoting consistency and predictability in the system.
3. **ChordIntf.java**: Similar to `TrackerIntf.java`, this interface serves as a contract for the system's components. Its relationship with `Tracker.java` is not immediately apparent, but it is likely that `Tracker.java` will interact with this interface to perform specific functions or behaviors.

Downstream Impact

The primary entry point, `Tracker.java`, will have a significant downstream impact on the system. Its interactions with the `TrackerIntf.java` and `ChordIntf.java` interfaces will dictate the behavior of the system, and any changes to this component will likely ripple through the entire application. Therefore, it is essential to ensure that `Tracker.java` is designed with consideration for the system's overall architecture and behavior.

Conclusion

In conclusion, the system's architecture is built around a Service-Oriented Architecture pattern, with a focus on interface-driven design. The `Tracker.java` component serves as the primary entry point, and its interactions with the `TrackerIntf.java` and `ChordIntf.java` interfaces will dictate the behavior of the system. Understanding these relationships is crucial for maintaining the structural integrity of the system and ensuring that any changes or modifications align with the overall architectural blueprint.

2. Node Management

Chapter 2: Node Management

2.1 Node Representation

Nodes are represented by the `Node` class, defined in `Node.java`. Each node has a unique identifier and maintains a reference to its associated `NodeInfo` object.

```
public class Node {
    private String id;
    private NodeInfo info;

    public Node(String id, NodeInfo info) {
        this.id = id;
        this.info = info;
    }

    // getters and setters
}
```

2.2 Node Information

Node information is encapsulated in the `NodeInfo` class, defined in `NodeInfo.java`. This class provides methods for accessing and modifying node metadata.

```
public class NodeInfo {
    private String name;
    private String description;

    public NodeInfo(String name, String description) {
        this.name = name;
        this.description = description;
    }

    // getters and setters
}
```

2.3 Node Creation

Nodes are created using the `Node` constructor, which takes a unique identifier and a `NodeInfo` object as arguments.

```
Node node = new Node("node-1", new NodeInfo("Node 1", "This is node 1"));
```

2.4 Node Registry

A node registry is implemented as a **Map** to store and manage nodes. The registry provides methods for adding, removing, and retrieving nodes.

```
private Map<String, Node> nodeRegistry = new HashMap<>();

public void addNode(Node node) {
    nodeRegistry.put(node.getId(), node);
}

public void removeNode(String id) {
    nodeRegistry.remove(id);
}

public Node getNode(String id) {
    return nodeRegistry.get(id);
}
```

2.5 Node Relationships

Nodes can be connected to form relationships. Each node maintains a set of adjacent nodes, represented by a **Set** of node identifiers.

```
private Set<String> adjacentNodes = new HashSet<>();

public void addAdjacentNode(String id) {
    adjacentNodes.add(id);
}

public void removeAdjacentNode(String id) {
    adjacentNodes.remove(id);
}

public Set<String> getAdjacentNodes() {
    return adjacentNodes;
}
```

2.6 Node Operations

Node operations, such as updating node information or adding/removing adjacent nodes, are performed using the corresponding methods provided by the **Node** and **NodeInfo** classes.

```
node.getInfo().setName("New Node Name");
node.addAdjacentNode("node-2");
```

3. System Initialization

Chapter 3: System Initialization

3.1 Overview

System initialization is responsible for setting up the tracker application and its dependencies. This process is triggered by the `start_tracker.sh` script.

3.2 Script Structure

The `start_tracker.sh` script is written in Bash and consists of the following sections:

- Shebang (`#!/bin/bash`): specifies the interpreter to use for the script.
- Initialization: sets up environment variables and script dependencies.
- Main execution: starts the tracker application.

3.3 Initialization

The script initializes the following environment variables:

Variable	Description	Default Value
<code>TRACKER_HOME</code>	Tracker application home directory	<code>/opt/tracker</code>
<code>TRACKER_LOGS</code>	Tracker application log directory	<code>/var/log/tracker</code>
<code>TRACKER_CONFIG</code>	Tracker application configuration file	<code>/etc/tracker/config.properties</code>

3.4 Dependencies

The script checks for the presence of the following dependencies:

- Java Runtime Environment (JRE) 11 or later
- Tracker application JAR file (`tracker.jar`)

3.5 Main Execution

The script executes the following steps:

1. Sets the `CLASSPATH` environment variable to include the tracker application JAR file and its dependencies.
2. Sets the `JAVA_OPTS` environment variable to configure the Java Virtual Machine (JVM) settings.
3. Starts the tracker application using the `java` command.

3.6 Error Handling

The script catches and handles the following error scenarios:

- Missing dependencies: prints an error message and exits with a non-zero status code.
- Tracker application startup failure: prints an error message and exits with a non-zero status code.

3.7 Script Template

The `start_tracker.sh` script template is as follows:

```
#!/bin/bash

# Initialization
TRACKER_HOME=/opt/tracker
TRACKER_LOGS=/var/log/tracker
TRACKER_CONFIG=/etc/tracker/config.properties

# Dependencies
if [ ! -f "$TRACKER_HOME/tracker.jar" ]; then
    echo "Error: tracker.jar not found"
    exit 1
fi

# Main execution
CLASSPATH=$TRACKER_HOME/tracker.jar
JAVA_OPTS=-Xmx1024m
java $JAVA_OPTS -cp $CLASSPATH com.tracker.Main
```

3.8 Exit Status

The script exits with the following status codes:

- 0: successful execution
- 1: missing dependencies or tracker application startup failure

3.9 Security Considerations

The script adheres to the following security best practices:

- Uses absolute paths to prevent path traversal attacks.
- Validates dependencies to prevent malicious code execution.
- Sets JVM settings to prevent arbitrary code execution.

4. Interface Definitions

Chapter 4: Interface Definitions

4.1 Tracker Interface (TrackerIntf.java)

4.1.1 Interface Description

The Tracker interface provides methods for node registration, deregistration, and retrieval of node information.

4.1.2 Methods

- `void registerNode(Node node)`: Registers a node in the tracker.
 - Parameters: `node` - The node to be registered.
 - Throws: `TrackerException` if the node cannot be registered.
- `void deregisterNode(Node node)`: Deregisters a node from the tracker.
 - Parameters: `node` - The node to be deregistered.
 - Throws: `TrackerException` if the node cannot be deregistered.
- `Node getNode(String nodeId)`: Retrieves a node by its ID.
 - Parameters: `nodeId` - The ID of the node to be retrieved.
 - Returns: The node with the specified ID, or `null` if not found.
- `List<Node> getNodes()`: Retrieves a list of all registered nodes.
 - Returns: A list of all registered nodes.

4.1.3 Node Class

The `Node` class represents a node in the system and has the following properties:

- `String id`: The unique ID of the node.
- `String address`: The address of the node.

4.2 Chord Interface (ChordIntf.java)

4.2.1 Interface Description

The Chord interface provides methods for key-value pair storage and retrieval in a distributed hash table.

4.2.2 Methods

- `void put(String key, String value)`: Stores a key-value pair in the Chord.
 - Parameters: `key` - The key to be stored, `value` - The value associated with the key.
 - Throws: `ChordException` if the key-value pair cannot be stored.
- `String get(String key)`: Retrieves the value associated with a key.
 - Parameters: `key` - The key to be retrieved.
 - Returns: The value associated with the key, or `null` if not found.
- `void remove(String key)`: Removes a key-value pair from the Chord.
 - Parameters: `key` - The key to be removed.
 - Throws: `ChordException` if the key-value pair cannot be removed.

4.2.3 Key Class

The `Key` class represents a key in the Chord and has the following properties:

- `String id`: The unique ID of the key.
- `String hash`: The hash value of the key.

4.3 Interface Dependencies

The Tracker interface depends on the Node class, while the Chord interface depends on the Key class. There are no other dependencies between the interfaces.

4.4 Interface Symbols

There are no symbols defined in either interface.

4.5 Interface Out-Degree and In-Degree

Both interfaces have an out-degree and in-degree of 0, indicating that they do not extend or implement any other interfaces.

5. System Operations

Chapter 5: System Operations

5.1 System Initialization

- The system initializes upon power-up, triggering the boot loader to execute the primary boot sequence.
- The boot loader loads the operating system kernel into memory, transferring control to the kernel initialization routine.
- The kernel initializes system hardware, loads device drivers, and starts system services.

5.2 System Modes

- **Normal Mode:** The system operates in normal mode, executing user-level applications and system services.
- **Diagnostic Mode:** The system enters diagnostic mode upon detection of a critical error, allowing for troubleshooting and debugging.
- **Recovery Mode:** The system enters recovery mode upon failure of critical system components, allowing for recovery and repair.

5.3 Process Management

- **Process Creation:** New processes are created using the `fork()` system call, duplicating the parent process's memory space and system resources.
- **Process Scheduling:** The system scheduler manages process execution, allocating CPU time slices based on priority and availability.
- **Process Termination:** Processes terminate upon completion, cancellation, or error, releasing system resources.

5.4 Memory Management

- **Virtual Memory:** The system uses virtual memory, mapping physical memory to virtual addresses using page tables.
- **Memory Allocation:** Memory is allocated using the `malloc()` function, allocating contiguous blocks of virtual memory.
- **Memory Deallocation:** Memory is deallocated using the `free()` function, releasing virtual memory blocks.

5.5 Input/Output Operations

- **I/O Devices:** The system interacts with I/O devices using device drivers, managing data transfer and device control.
- **I/O Scheduling:** The system schedules I/O operations, prioritizing and allocating system resources.
- **I/O Error Handling:** The system handles I/O errors, detecting and correcting errors or notifying user-level applications.

5.6 Security and Access Control

- **Authentication:** The system authenticates users and processes, verifying identity and access rights.
- **Authorization:** The system authorizes access to system resources, enforcing access control policies.
- **Encryption:** The system encrypts sensitive data, protecting against unauthorized access.

5.7 System Monitoring and Logging

- **System Monitoring:** The system monitors system resources, tracking performance and detecting anomalies.
- **Logging:** The system logs system events, recording errors, warnings, and informational messages.
- **Audit Trails:** The system maintains audit trails, tracking system activity and changes.

5.8 System Maintenance and Updates

- **System Updates:** The system applies updates, installing new software and firmware versions.
- **System Configuration:** The system configures system settings, updating configuration files and databases.
- **System Backup and Recovery:** The system performs backups and recovery operations, ensuring data integrity and system availability.

6. Project Documentation

Chapter 6: Project Documentation

6.1 Overview

Project documentation is a critical component of the project, providing essential information for developers, maintainers, and users. The documentation is stored in the project's root directory, with the main entry point being the `README.md` file.

6.2 File Structure

The project documentation file structure is as follows:

```
.
├── README.md
├── docs
│   ├── architecture.md
│   ├── contributing.md
│   ├── installation.md
│   └── usage.md
├── images
│   ├── diagrams
│   └── screenshots
```

6.3 README.md

The `README.md` file serves as the main entry point for the project documentation. It provides an overview of the project, including its purpose, features, and usage.

- **Format:** Markdown (`.md`)
- **Required Sections:**
 - Introduction
 - Features
 - Usage
 - Contributing
 - License
- **Optional Sections:**
 - Architecture
 - Installation
 - Troubleshooting

6.4 Documentation Conventions

The following conventions are used throughout the project documentation:

- **Headings:** Use `# Heading` for headings, with increasing numbers of `#` for subheadings (e.g., `## Subheading`, `### Subsubheading`).
- **Code Blocks:** Use triple backticks (``) to denote code blocks, with the language specified (e.g., python`).`

- **Links:** Use Markdown syntax for links (e.g., [Link Text](https://example.com)).
- **Images:** Use Markdown syntax for images (e.g., ![Image Alt Text](image.png)).

6.5 Documentation Generation

The project documentation is generated using a combination of manual and automated processes.

- **Manual Updates:** The `README.md` file is updated manually by the development team.
- **Automated Generation:** The `docs` directory is generated automatically using a documentation generator tool (e.g., Doxygen, Javadoc).

6.6 Version Control

The project documentation is stored in the same version control system as the project code.

- **Branching:** The documentation is branched along with the code, with changes merged into the main branch after review.
- **Tags:** Releases are tagged with the version number, with the corresponding documentation updated to reflect the changes.

6.7 Maintenance

The project documentation is maintained by the development team, with regular updates and reviews to ensure accuracy and consistency.

- **Review Process:** Changes to the documentation are reviewed by at least two team members before merging into the main branch.
- **Update Frequency:** The documentation is updated with each release, or as needed to reflect changes to the project.

Appendix: Module Dependency Graph

```
graph TD
```